

Setor de Educação Profissional e Tecnológica Tecnologia em Análise e Desenvolvimento de Sistemas

DS122 - Desenvolvimento de Aplicações Web 1

05 - CSS3: fundamentos

Prof. Alex Kutzke

Agosto de 2026

Objetivos da aula

Ao final desta aula você deve ser capaz de:

1. Ligar uma folha de estilo externa ao documento;
2. Escrever seletores de tipo, classe, id, descendente, atributo e pseudo-classe;
3. Prever qual declaração vence um conflito;
4. Calcular a largura que um elemento ocupa na tela;
5. Escolher entre px, rem, em e %;
6. Declarar cores e verificar o contraste com o fundo;
7. Definir a tipografia da página;
8. Descobrir no inspetor por que uma regra não foi aplicada.

Roteiro

Fundamentos

Seletores

Cascata

Caixa

Unidades e cores

Tipografia

Prática

A página que já tinha aparência

A página da aula passada não tem CSS nenhum. Mesmo assim:

- o `<h1>` aparece grande e em negrito;
- os itens de `<ul>` aparecem com marcador;
- o link aparece azul e sublinhado.

Isso vem da **folha de estilo do agente de usuário**, que o navegador aplica a todo documento para que ele seja legível sem CSS.

No inspetor (F12), as regras dela aparecem marcadas como *user agent stylesheet*.

Escrever CSS é substituir aquelas decisões

O que muda com o CSS

O navegador continua aplicando a folha dele para tudo o que você não disser. Cada regra sua substitui uma decisão daquela folha.

CSS: *Cascading Style Sheets*. A linguagem descreve a **apresentação** de um documento HTML.

Estrutura e significado ficam no HTML. Aparência fica no CSS. Comportamento fica no JavaScript, na segunda parte do semestre.

Onde o CSS entra: arquivo externo

A forma usada na disciplina e em produção:

```
<head>
  <meta charset="utf-8">
  <title>Catálogo</title>
  <link rel="stylesheet" href="css/estilo.css">
</head>
```

- o arquivo .css não tem nenhuma tag HTML dentro;
- o href segue as regras de caminho relativo já conhecidas;
- um arquivo atende o site inteiro, e fica em cache.

As outras duas formas

Elemento `<style>` no documento, para teste rápido:

```
<style>
  p { color: #333333; }
</style>
```

Atributo `style` na tag:

```
<p style="color: red;">Texto</p>
```

O atributo mistura apresentação e estrutura, obriga a repetir a declaração em cada elemento e vence qualquer seletor da folha. Voltaremos a ele na especificidade.

Anatomia de uma regra

```
/* isto é um comentário, e é a única forma de comentar */
h1 {
  color: #1a3d6d;
  font-size: 2rem;
}
```

- h1: o **seletor**, diz a quais elementos a regra se aplica;
- entre chaves: o **bloco de declarações**;
- color: #1a3d6d;: uma **declaração**, com **propriedade**, dois pontos, **valor** e ponto e vírgula.

CSS não avisa quando você erra

Falha silenciosa

Propriedade inexistente, valor inválido, chave faltando: o navegador descarta o que não entendeu e segue adiante. Nenhuma mensagem aparece na tela.

Consequências para o dia a dia:

- “não funcionou” quase nunca significa que o navegador está errado;
- o painel **Styles** do inspetor mostra o que foi aplicado e o que foi descartado;
- o validador do W3C aponta o nome digitado errado.

Tipo, classe e id

```
/* tipo: todos os parágrafos */
```

```
p { line-height: 1.6; }
```

```
/* classe: todo elemento com class="destaque" */
```

```
.destaque { background-color: #fff3cd; }
```

```
/* id: o único elemento com id="rodape" */
```

```
#rodape { font-size: 0.875rem; }
```

A classe se repete à vontade no documento. O id é único.

Universal, agrupamento e relação

```
/* universal: qualquer elemento */
* { box-sizing: border-box; }

/* agrupamento: a mesma declaração para vários seletores */
h1, h2, h3 { font-family: Georgia, serif; }

/* descendente: um <a> em qualquer nível dentro do <nav> */
nav a { text-decoration: none; }

/* filho direto: um <li> filho imediato do <ul> */
ul > li { margin-bottom: 0.5rem; }
```

O espaço entre dois seletores significa “dentro de”.

Atributo, pseudo-classe e pseudo-elemento

```
/* atributo */  
input[type="text"] { border: 1px solid #999999; }  
  
/* pseudo-classe: estado do elemento */  
a:hover { text-decoration: underline; }  
a:focus-visible { outline: 3px solid #1a73e8; }  
  
/* pseudo-elemento: parte do elemento, sem tag própria */  
p::first-line { font-variant: small-caps; }
```

Pseudo-classe: dois pontos, descreve estado. Pseudo-elemento: dois sinais de dois pontos, alcança um pedaço do elemento.

O contorno de foco não se apaga

Acessibilidade

Quem navega por teclado enxerga onde está pelo contorno de foco. `outline: none` deixa a pessoa perdida na página.

Se o contorno padrão não combina com o visual, troque a aparência dele:

```
a:focus-visible {  
  outline: 3px solid #1a73e8;  
  outline-offset: 2px;  
}
```

Classe ou id?

Classe

- reutilizável;
- quantos elementos você quiser;
- escolha padrão para estilo.

id

- único no documento;
- alvo de âncora
(`href="#rodape"`);
- usado pelo JavaScript.

Para estilo, prefira classe. O `id` tem especificidade alta demais, e a regra presa a ele fica difícil de sobrescrever depois. O próximo bloco mostra isso em números.

Herança

```
body {  
  font-family: Verdana, sans-serif;  
  color: #222222;  
}
```

Todo o texto da página aparece em Verdana e cinza escuro, sem nenhuma regra para p, li ou td.

Algumas propriedades passam do elemento para os descendentes. Outras não.

O que herda e o que não herda

Herdam	Não herdam
color	margin
font-family	padding
font-size	border
line-height	background
text-align	width, height

Propriedades de texto são herdadas. Propriedades de caixa não são, e é o comportamento desejado: um padding no body não vira padding em cada parágrafo.

A página da propriedade na MDN traz a linha *Herdada: sim ou não*.

A cascata resolve conflitos em três passos

Duas regras dizem coisas diferentes sobre a mesma propriedade do mesmo elemento. O navegador decide nesta ordem:

1. **Origem e importância:** o autor da página vence o navegador; `!important` sobe acima de todas as declarações normais;
2. **Especificidade:** vence o seletor mais específico;
3. **Ordem de aparição:** no empate, vence a última.

O passo 3 explica por que a ordem das regras no arquivo importa, e por que a folha de um framework vem antes da sua no `<head>`.

Especificidade: contagem em três casas

Seletor	A	B	C
p	0	0	1
.texto	0	1	0
p.texto	0	1	1
.artigo p.destaque	0	2	1
#noticia p	1	0	1
input[type="text"]:focus	0	2	1

A: id. **B:** classes, atributos e pseudo-classes. **C:** tipos e pseudo-elementos. O * não conta nada.

Compara-se da esquerda para a direita: um id vence qualquer quantidade de classes.

Qual cor vence?

```
<div class="artigo" id="noticia">  
  <p class="texto destaque">O sistema entra em produção.</p>  
</div>
```

p	{ color: blue; }
.texto	{ color: green; }
.destaque	{ color: orange; }
.artigo p.destaque	{ color: purple; }
#noticia p	{ color: red; }

Vermelho

#noticia p tem A igual a 1, e nenhuma quantidade de classes alcança isso.

Duas observações sobre o mesmo exemplo:

- entre .texto e .destaque, que empatam em (0,1,0), venceria .destaque, por vir depois no arquivo;
- a ordem das classes dentro do atributo class do HTML não influencia nada.

Acima de tudo: o atributo style, e o !important

O atributo `style` na tag vence qualquer seletor. É por isso que o estilo *inline* atrapalha: ele tira de você a chance de corrigir aquela declaração pela folha.

Sobre o !important

Ele funciona, e cria um problema maior do que resolveu: a única forma de vencer um `!important` depois é outro `!important`. Considere-o proibido nesta disciplina.

Quando der vontade de usá-lo, o que falta é descobrir por que a sua regra perdeu. Faremos isso no inspetor.

Seletor simples é seletor que dá para corrigir

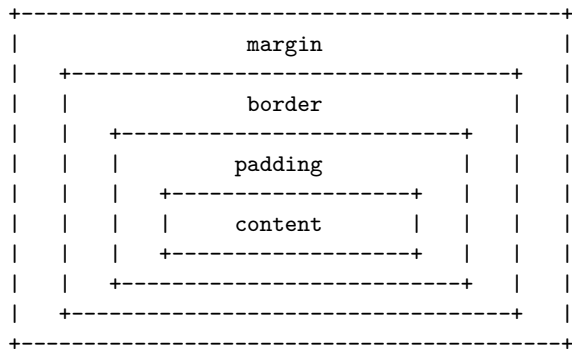
Regra de bolso

Escreva o seletor mais simples que resolve o problema.

Uma folha construída sobre classes de um nível é fácil de ajustar depois.

Uma folha cheia de `#menu .lista li a span` obriga cada correção futura a ser mais específica que a anterior, e o arquivo cresce em espiral.

Todo elemento é uma caixa



padding recebe a cor de fundo. margin é sempre transparente.

Formas abreviadas, no sentido horário

<code>padding: 10px;</code>	<i>/* os quatro lados */</i>
<code>padding: 10px 20px;</code>	<i>/* topo e base direita e</i>
<code>↪ esquerda */</code>	
<code>padding: 10px 20px 5px;</code>	<i>/* topo direita e esquerda </i>
<code>↪ base */</code>	
<code>padding: 10px 20px 5px 0;</code>	<i>/* topo direita base </i>
<code>↪ esquerda */</code>	
<code>padding-left: 20px;</code>	<i>/* um lado só */</i>
<code>border: 5px solid #000000;</code>	<i>/* espessura, estilo e cor */</i>

As mesmas formas valem para `margin` e para `border-width`.

A largura ocupada pela caixa

```
.caixa {  
  width: 300px;  
  padding: 30px;  
  border: 5px solid #000000;  
}
```

Largura ocupada na tela:

$$300 + 30 + 30 + 5 + 5 = \mathbf{370px}$$

width dimensiona apenas o content. Duas colunas de width: 50% com padding somam mais que a largura disponível, e a segunda desce para baixo da primeira.

box-sizing: border-box

```
.caixa {  
  box-sizing: border-box;  
  width: 300px;  
  padding: 30px;  
  border: 5px solid #000000;  
}
```

Agora a caixa ocupa **300px**, e o conteúdo é espremido para $300 - 60 - 10 = 230\text{px}$.

Por isso a linha abaixo abre praticamente toda folha de estilo moderna:

```
* { box-sizing: border-box; }
```

Colapso de margens verticais

Primeira caixa com `margin-bottom`: 30px. Segunda com `margin-top`: 20px. O espaço entre elas é de **30px**.

Margens verticais adjacentes se fundem, e a maior prevalece.

- acontece também entre um elemento e o pai, quando não há borda nem padding separando os dois;
- margens horizontais nunca colapsam;
- dentro de Flexbox e Grid não há colapso, o que é um dos motivos de esses modelos deixarem o espaçamento previsível.

Saída prática: espaçar sempre para o mesmo lado, usando só a margem de baixo entre irmãos.

display

Valor	Ocupa a linha	Aceita width	Margem vertical
block	sim	sim	empurra
inline	não	não	não empurra
inline-block	não	sim	empurra
none	o elemento some da página		

`<div>`, `<p>`, `<h1>` e `<section>` são block. `<a>`, `<span>`, `<strong>` e `<img>` são inline.

`display: none` também tira o elemento do leitor de tela.

Link vira botão com inline-block

```
.botao {  
  display: inline-block;  
  padding: 0.75rem 1.5rem;  
  background-color: #1a3d6d;  
  color: #ffffff;  
  text-decoration: none;  
}
```

Sem o `inline-block`, o `padding` vertical não afasta as linhas vizinhas e `width` é ignorado, porque `<a>` é `inline`.

Unidades de medida

```
.exemplo {  
  width: 300px;      /* pixel CSS, medida absoluta */  
  width: 50%;        /* relativo ao elemento pai */  
  font-size: 1rem;   /* relativo à fonte do <html> */  
  padding: 0.5em;    /* relativo à fonte do próprio elemento */  
  height: 100vh;     /* 100% da altura da janela */  
  width: 80vw;       /* 80% da largura da janela */  
}
```

Quando usar cada uma

- px: bordas e detalhes que não devem escalar;
- rem: texto e espaçamento. Apoia-se na fonte do `<html>`, que é a preferência configurada por quem lê;
- em: se apoia na fonte do próprio elemento, e por isso acumula em elementos aninhados. Útil no padding de botão;
- %: depende da propriedade. Em `width`, é a largura do pai;
- vw e vh: centésimos da janela, para seções de tela cheia.

Não fixe o tamanho de fonte do html

Linha a evitar

```
html { font-size: 16px; }
```

Ela aparece em muito material antigo e anula o ajuste de tamanho de fonte de quem enxerga pouco, uma das configurações de acessibilidade mais usadas.

O padrão do `<html>` já é a preferência da pessoa. Para deixar isso explícito:

```
html { font-size: 100%; }
```

Notações de cor

```
color: red;                /* palavra-chave */
color: #1a3d6d;            /* hexadecimal RR GG BB */
color: #fff;               /* forma curta de #ffffff */
color: rgb(26, 61, 109);    /* 0 a 255 em cada canal */
color: rgba(26, 61, 109, 0.5); /* o quarto valor é a opacidade
↪ */
color: hsl(212, 61%, 26%);  /* matiz, saturação,
↪ luminosidade */
```

O hexadecimal é o que aparece em qualquer paleta pronta. O `hsl()` é o mais fácil de ajustar à mão: para o azul mais escuro do `:hover`, basta reduzir o terceiro valor.

Contraste mínimo entre texto e fundo

Texto cinza claro sobre fundo branco fica ilegível para quem tem baixa visão, para quem usa a tela sob luz forte e para parte das pessoas com daltonismo.

A WCAG define a razão mínima de contraste entre texto e fundo:

- **4.5:1** para texto normal;
- **3:1** para texto grande, a partir de 24px, ou 18.66px em negrito.

Confira em <https://webaim.org/resources/contrastchecker/>, colando os dois hexadecimais. O inspetor mostra a mesma razão ao lado da cor.

ODS 4 e ODS 10: contraste insuficiente exclui parte do público sem que o autor perceba: no monitor dele, o texto está legível.

font-family recebe uma lista

```
body {  
  font-family: "Segoe UI", Roboto, Helvetica, Arial, sans-serif;  
}
```

O navegador tenta a primeira, e passa à seguinte se ela não estiver instalada na máquina.

- a lista termina em família genérica: `sans-serif`, `serif` ou `monospace`;
- nomes com espaço vão entre aspas.

Tamanho, peso e altura de linha

```
body { font-size: 1rem; line-height: 1.6; }  
h1   { font-size: 2.5rem; font-weight: 700; }
```

`line-height` **sem unidade**: o valor é herdado como fator, e cada elemento o multiplica pelo próprio tamanho de fonte. Entre 1.4 e 1.6 para texto corrido.

`font-weight`: `normal` é 400, `bold` é 700, e os múltiplos de 100 valem quando a fonte tem esses cortes.

Fonte externa do Google Fonts

No <head>, antes da sua folha:

```
<link rel="preconnect" href="https://fonts.googleapis.com">
<link href="https://fonts.googleapis.com/css2?family=Roboto:wght@400;700&display=swap" rel="stylesheet">
```

No CSS, mantendo a pilha de reserva:

```
body { font-family: Roboto, Arial, sans-serif; }
```

Cada peso pedido é um arquivo a mais para baixar. Peça dois, no máximo três.

Roteiro da prática

Trabalhando sobre as páginas da tarefa de HTML:

1. criar `css/estilo.css` e ligá-lo pelo `<link>`;
2. descobrir no inspetor por que uma regra não pegou;
3. medir a caixa no diagrama do painel **Computed**;
4. verificar contraste e validar a folha no W3C.

O texto completo do roteiro está no material da aula, na parte prática.

Por que a regra não pegou

Com `class="menu"` no `<nav>`, e estas três regras nesta ordem:

```
nav a { color: #1a3d6d; }  
a     { color: #cc0000; }  
.menu a { color: #008000; }
```

Decida a cor antes de recarregar. Depois, no painel **Styles**:

- as declarações perdedoras aparecem **riscadas**;
- ao lado de cada regra, o arquivo e a linha onde ela está;
- a aba **Computed** mostra o valor final já resolvido.

Medir a caixa

```
.caixa {  
  width: 300px; padding: 30px;  
  border: 5px solid #000000;  
  background-color: #ffd400;  
}  
.corrigida { box-sizing: border-box; }
```

No fim do painel **Computed**, o diagrama do modelo de caixa traz os quatro valores. Confirme: **370px** na primeira, **300px** na segunda.

Exercícios

Entrega

Valem nota, no item *Exercícios em sala*. Prazo: **03/09, às 23h59**, por push no GitLab.

- enunciado no README.md do repositório-modelo, com o endereço na UFPR Virtual;
- individual ou em dupla, com *fork* dentro do grupo;
- atividade avaliativa: sem IA generativa para produzir o código;
- a folha que sair daqui recebe o layout no próximo encontro e vai na Entrega 1 do trabalho, em 11/09.

O que ainda falta para montar um layout

Você controla a aparência de cada caixa. Falta posicionar as caixas umas em relação às outras: menu na horizontal, três cartões lado a lado, coluna lateral acompanhando a altura do conteúdo.

Próximo encontro: Flexbox, Grid e *media queries*.

Os fundamentos de hoje continuam valendo lá dentro: a caixa que o Flexbox posiciona é a mesma caixa desta aula.

Resumo

- toda página já vem estilizada pela folha do navegador;
- conflito se resolve por origem, especificidade e ordem, nessa sequência;
- um id vence qualquer quantidade de classes; prefira classe;
- propriedades de texto são herdadas, as de caixa não;
- width dimensiona só o conteúdo, até você escrever `box-sizing: border-box;`
- margens verticais adjacentes colapsam;
- rem para texto, e sem fixar a fonte do `<html>`;
- contraste de 4.5:1 é requisito;
- CSS inválido não gera erro na tela: a resposta está no inspetor.